HASHING

Unit 4: Direct-Address Tables, Hash Tables, Open Addressing, Perfect Hashing

Notes for the written exam. Learn the definitions, the formulas and the solved examples.

1. Introduction to Hashing

To search for a value in a list we can use linear search, which takes O(n) time, or binary search on sorted data, which takes O(log n) time. Hashing tries to do the search in **O(1)** time on average, that is, in about one step, however large the data is.

Definition: Hashing is a technique in which the position of a key in a table is **calculated from the key itself** by using a function called the hash function. The key can then be inserted, searched and deleted by going directly to that position.

Working: **Key -> Hash function -> Index (slot number) -> Table**

Terms used in hashing

Term	Meaning
Key	The value that we want to store or search (for example roll number 54).
Universe of keys (U)	The set of all possible keys.
Hash function h(k)	A function that converts the key k into a table index.
Hash table T	The array in which the keys are stored. Its size is m.
Slot (or bucket)	One position of the hash table.
Hash value	The index given by the hash function, from 0 to m - 1.
Collision	The situation when two different keys give the same hash value.
Load factor (alpha)	alpha = n / m, where n = number of keys stored and m = number of slots.

Operations: insert, search and delete. In hashing all three take O(1) time on average.

2. Direct-Address Tables

Definition: A **direct-address table** is an array T[0 .. M-1] in which the **key itself is used as the index**. The key k is stored in slot T[k]. It is used when the keys are small integers taken from a small universe U = {0, 1, ..., M-1} and no two keys are equal.

How it works

1. Make an array T with one slot for every possible key (M slots).
2. **Insert** key k: store it (or a pointer to its record) in T[k].
3. **Search** key k: look at T[k]. If it is not empty, the key is present; otherwise it is absent.
4. **Delete** key k: make T[k] empty (NIL).

All three operations take **O(1) time in the worst case**, because we go straight to the slot.

Example

Universe U = {0, 1, 2, ..., 9}. We want to store the keys 2, 3, 5 and 8.

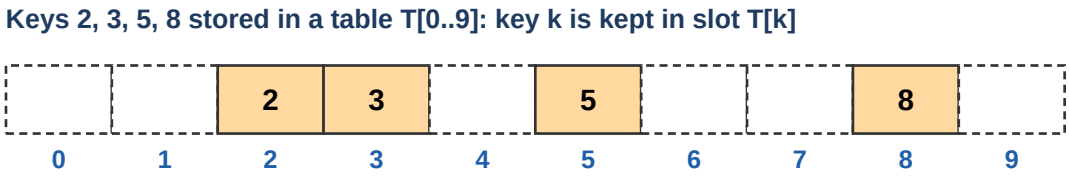

Figure 1: A direct-address table for the keys 2, 3, 5 and 8

To search for 5, look at T[5]: it is filled, so 5 is present. To search for 4, look at T[4]: it is empty, so 4 is absent.

Advantages

1. Very simple to understand and implement.
2. Search, insert and delete take O(1) time in the worst case.
3. There are no collisions, because every key has its own slot.

Disadvantages

1. The table needs one slot for **every possible key**. If the universe is large, a huge amount of memory is wasted. For example, to store 100 ten-digit phone numbers we would need 10 billion slots.
2. It works only for keys that are integers (or can be converted to small integers).
3. If the universe is very large, the table may not fit in memory at all.

Note: Because of these problems we use a smaller table and a hash function. This gives the **hash table**.

3. Hash Tables

Definition: A **hash table** is a data structure that stores keys in a table of size m using a **hash function** h that maps every key to a slot number. The table size m is much smaller than the number of possible keys, so memory is saved.

The hash function is h : U -> {0, 1, ..., m-1}. Key k is stored in slot h(k), and we say that k **hashes to** slot h(k).

Characteristics of a good hash function

1. It should be **easy and fast** to calculate.
2. It should give the **same hash value every time** for the same key.
3. It should **spread the keys evenly** over the table (uniform distribution).
4. It should cause **as few collisions** as possible.
5. It should use the **whole key**, not only a part of it.

Common hash functions

(a) Division method. h(k) = k mod m. The key is divided by the table size and the remainder is the slot number. The value of m should be a **prime number** and not a power of 2 (a power of 2 would use only the last few bits of the key).

Example: m = 11. For k = 54, h(54) = 54 mod 11 = 10. Similarly h(26) = 4, h(93) = 5, h(17) = 6, h(77) = 0, h(31) = 9.

(b) Multiplication method. Steps: (1) multiply the key k by a constant A, where 0 < A < 1; (2) take the fractional part of the result; (3) multiply it by m and take the floor. So h(k) = floor(m x (kA mod 1)). A good choice is A = 0.618 (the golden ratio value). The advantage is that the value of m is not critical.

Example: k = 21, m = 10, A = 0.618. kA = 12.978, fractional part = 0.978, m x 0.978 = 9.78, so h(21) = **9**.

(c) Mid-square method. Square the key and take the middle digits as the slot number. Example: k = 123, k^2 = 15129, middle two digits = 51, so h(123) = 51 (for a table of size 100).

(d) Folding method. Divide the key into equal parts, add the parts, and take the last digits of the sum. Example: k = 123456789. Parts 123, 456, 789. Sum = 1368. Last three digits give h(k) = **368** (for a table of size 1000).

Collision

Definition: A **collision** occurs when two different keys k1 and k2 have the same hash value, that is h(k1) = h(k2), so both want the same slot.

Collisions cannot be avoided completely, because the number of possible keys is much larger than the number of slots. So we must have a method to handle them. The two main methods are **chaining** and **open addressing**.

Collision resolution by chaining

Definition: In **chaining** (separate chaining) every slot of the hash table points to a **linked list**. All the keys that hash to the same slot are kept in the list of that slot.

1. **Insert:** find slot h(k) and add the key at the beginning of its list. Time O(1).
2. **Search:** find slot h(k) and search the list for the key. Time O(1 + alpha) on average.
3. **Delete:** find the key in the list of slot h(k) and remove the node.

Example: m = 7, h(k) = k mod 7. Insert the keys 50, 700, 76, 85, 92, 73, 101. The hash values are 1, 0, 6, 1, 1, 3, 3.

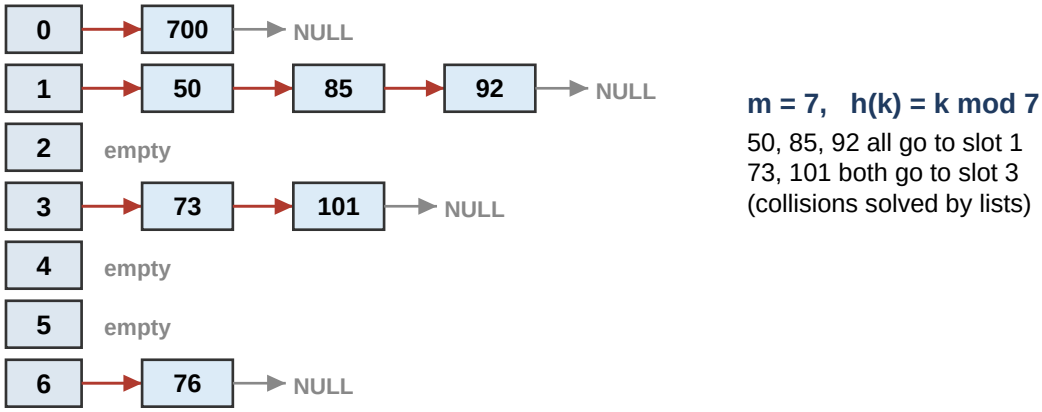

Figure 2: Hash table with chaining (m = 7)

Analysis. Let load factor alpha = n/m. This is the average length of a list. A search takes O(1 + alpha) time on average. If alpha is kept at most about 1, the search takes O(1) on average. In the worst case all the keys fall in one list and the search takes O(n).

Advantages of chaining	Disadvantages of chaining
Simple to implement	Extra memory is needed for the pointers
The table never becomes full; alpha can be more than 1	Lists may become long if the hash function is poor
Deletion is easy	Linked lists are slower to traverse than an array
Performance falls slowly as alpha increases	Needs extra space outside the table

Load factor and rehashing

When the load factor becomes too high, the hash table becomes slow. The remedy is **rehashing**: a new table of about double the size (preferably a prime number) is created, and all the existing keys are inserted again using the new table size. Rehashing takes O(n) time but it is done rarely.

4. Open Addressing

Definition: In **open addressing** all the keys are stored **inside the hash table itself**. There are no linked lists. If the slot given by the hash function is already occupied, we look for another empty slot by following a fixed rule. This is called **probing**.

- Because every key takes one slot, the number of keys can never be more than the table size: n <= m, so alpha <= 1.
- The hash function now has two inputs, the key and the probe number i: h(k, i), where i = 0, 1, 2, ..., m-1.
- The sequence of slots h(k,0), h(k,1), h(k,2), ... is called the **probe sequence** of the key.

Operations

- **Insert:** check slot h(k,0). If it is occupied, check h(k,1), then h(k,2), and so on, until an empty slot is found. Put the key there. If no empty slot is found, the table is full.
- **Search:** follow the same probe sequence. If the key is found, the search is successful. If an **empty slot** is reached, the key is not present (unsuccessful search).
- **Delete:** the slot is **not** simply made empty. It is marked **DELETED**.

Why a deleted slot is marked DELETED

Take m = 10 and linear probing. The keys 23, 43 and 13 all hash to slot 3, so they are stored in slots 3, 4 and 5. Now delete 43 and make slot 4 empty. If we search for 13 now, we check slot 3 (23, not equal), then slot 4, which is empty, so we stop and wrongly say that 13 is absent. If slot 4 is marked DELETED, the search goes on to slot 5 and finds 13. So during search a DELETED slot is skipped, and during insertion a DELETED slot can be reused.

4.1 Linear probing

Definition: In **linear probing** the next slots are tried one after another in order: h(k, i) = (h'(k) + i) mod m, for i = 0, 1, 2, ..., m-1, where h'(k) is an ordinary hash function such as k mod m. After the last slot we return to slot 0 (wrap around).

Example. m = 10, h'(k) = k mod 10. Insert 89, 18, 49, 58, 69.

Key	h'(k)	Slots tried	Final slot
89	9	9 (free)	9
18	8	8 (free)	8
49	9	9 (taken), 0 (free)	0
58	8	8, 9, 0 (taken), 1 (free)	1
69	9	9, 0, 1 (taken), 2 (free)	2

0	1	2	3	4	5	6	7	8	9
49	**58**	**69**						**18**	**89**

First row = slot number, second row = key stored (blank = empty)

Advantages	Disadvantages
Very simple to implement	**Primary clustering:** filled slots join together to form long runs
All slots can be reached; no slot is missed	A key hashing into a cluster has to probe to its end, and the cluster grows even bigger
Good use of the cache because slots are next to each other	Search time increases quickly as the table fills

4.2 Quadratic probing

Definition: In **quadratic probing** the gap between the slots tried increases as a square: h(k, i) = (h'(k) + c1 i + c2 i^2) mod m, where c1 and c2 are constants and c2 is not 0. In the simple form h(k, i) = (h'(k) + i^2) mod m the slots tried are h', h'+1, h'+4, h'+9, ...

Example. m = 10, h(k, i) = (k mod 10 + i^2) mod 10. Insert 89, 18, 49, 58, 69.

Key	h'(k)	Slots tried	Final slot
89	9	9 (free)	9
18	8	8 (free)	8
49	9	9 (taken), 9+1 = 10 gives 0 (free)	0
58	8	8 (taken), 8+1 = 9 (taken), 8+4 = 12 gives 2 (free)	2
69	9	9 (taken), 9+1 = 0 (taken), 9+4 = 13 gives 3 (free)	3

0	1	2	3	4	5	6	7	8	9
49		**58**	**69**					**18**	**89**

First row = slot number, second row = key stored (blank = empty)

- Quadratic probing **removes primary clustering**, because the keys are spread out by bigger jumps.
- **Secondary clustering** remains: two keys with the same starting slot follow the same probe sequence.
- It may not reach all the slots. For m = 10 the values of i^2 mod 10 are only 0, 1, 4, 5, 6, 9, so only 6 of the 10 slots are ever tried.

- If m is a **prime number** and the table is **less than half full**, quadratic probing always finds an empty slot.

4.3 Double hashing

Definition: In **double hashing** two hash functions are used. The first gives the starting slot and the second gives the step size: h(k, i) = (h1(k) + i x h2(k)) mod m. The step size depends on the key.

- h2(k) must **never be 0**, otherwise the same slot is probed again and again.
- h2(k) should be **relatively prime to m** so that all slots can be reached. This is easy if m is a prime number.
- Common choice: h1(k) = k mod m and h2(k) = 1 + (k mod m'), where m' is a little smaller than m.

Example. m = 11, h1(k) = k mod 11, h2(k) = 1 + (k mod 10). Insert 89, 18, 49, 58, 69.

Key	h1(k)	h2(k)	Slots tried	Final slot
89	1	10	1 (free)	1
18	7	9	7 (free)	7
49	5	10	5 (free)	5
58	3	9	3 (free)	3
69	3	10	3 (taken), (3 + 10) mod 11 = 2 (free)	2

0	1	2	3	4	5	6	7	8	9	10
	89	**69**	**58**		**49**		**18**			

First row = slot number, second row = key stored (blank = empty)

Advantages	Disadvantages
Almost no clustering (neither primary nor secondary)	Two hash functions must be calculated, so it is slower per probe
Many different probe sequences, so it behaves close to ideal hashing	The second function must be chosen carefully
Best of the three probing methods for performance	Cache performance is poorer because the jumps are large

4.4 Comparison of the three probing methods

Point	Linear probing	Quadratic probing	Double hashing
Formula	(h'(k) + i) mod m	(h'(k) + c1 i + c2 i^2) mod m	(h1(k) + i h2(k)) mod m
Step size	Always 1	Increases as a square	Depends on the key
Primary clustering	Yes	No	No
Secondary clustering	Yes	Yes	No
Number of different probe sequences	m	m	About m^2
Implementation	Easiest	Easy	Slightly harder
Performance	Poorest	Better	Best

4.5 Performance of open addressing

Assume that each key is equally likely to follow any probe sequence (uniform hashing) and alpha = n/m is less than 1. The expected number of probes is:

- Unsuccessful search (and insertion): at most **1 / (1 - alpha)**.
- Successful search: at most **(1 / alpha) x ln(1 / (1 - alpha))**.

Load factor alpha	Unsuccessful search	Successful search
0.5	2 probes	1.39 probes
0.75	4 probes	1.85 probes
0.9	10 probes	2.56 probes

So the table should not be allowed to become nearly full. Rehash when alpha crosses about 0.7.

4.6 Chaining vs open addressing

Point	Chaining	Open addressing
Where the keys are stored	In linked lists outside the table	Inside the table itself
Load factor	Can be more than 1	Must be 1 or less
Extra memory	Needed for pointers	Not needed
Deletion	Easy	Difficult (DELETED mark needed)
Clustering	No clustering problem	Clustering can occur
Behaviour when the table is nearly full	Slows down gradually	Slows down sharply
Best when	Number of keys is not known in advance	Number of keys is known and memory is limited

5. Perfect Hashing

Definition: A **perfect hash function** is a hash function that maps the keys to the table slots with **no collision at all**. **Perfect hashing** is a technique that gives **O(1) search time in the worst case** (not only on average).

It can be used only when the set of keys is **static**: the keys are known in advance and no key is inserted or deleted later. Examples are the reserved words of a programming language, the names of the commands of a system, or a fixed dictionary stored on a CD-ROM.

Two-level scheme (idea)

Perfect hashing uses **two levels** of hashing. At the first level the keys are hashed into a table, and where several keys fall in the same slot, a second small hash table is built for that slot, big enough so that there is no collision in it.

Steps

- **Level 1:** Take n keys. Create a table of size m = n. Hash the keys into it using a hash function h. Some slots may get more than one key. Let slot j get n_j keys.
- **Level 2:** For every slot j create a **secondary table** S_j of size $\mathbf{n_j^2}$ (the square of the number of keys in that slot). Use a separate hash function h_j for it, chosen so that it gives **no collision** among the n_j keys. If it gives a collision, choose another function and try again.
- A slot with no key needs no secondary table. A slot with one key has a table of size 1.
- **Search for key k:** calculate j = h(k) and go to slot j. Then calculate $h_j(k)$ and look at that position in S_j. Only two hash calculations are needed, so the worst case time is O(1).

Example

Keys: 12, 25, 31, 44 (n = 4). Level 1: m = 4 and h(k) = k mod 4. The hash values are 12 gives 0, 25 gives 1, 31 gives 3, 44 gives 0. So slot 0 has 2 keys (a collision), slots 1 and 3 have one key each, and slot 2 has none.

Slot 0 has $n_0 = 2$ keys, so its secondary table has size $2^2 = 4$. Take $h_0(k) = (k \bmod 5) \bmod 4$. Then $h_0(12) = 2 \bmod 4 = 2$ and $h_0(44) = 4 \bmod 4 = 0$. The two values are different, so there is no collision.

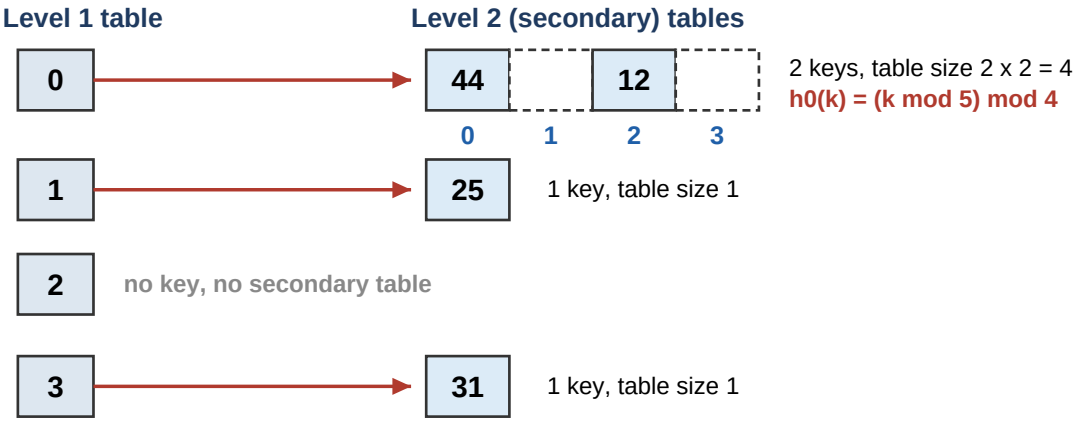

Figure 3: Perfect hashing with two levels for the keys 12, 25, 31, 44

To search for 44: h(44) = 44 mod 4 = 0, so go to slot 0. Then $h_0(44) = 0$, so look at position 0 of the secondary table, where 44 is found.

Why is the secondary table size the square of the number of keys?

If n_j keys are put in a table of size n_j^2 using a randomly chosen hash function, the probability that there is any collision is **less than 1/2**. So at least half of the possible functions work, and we find a good one after about 2 tries on average. A smaller table would make collisions likely.

Space

The total space is the size of the first table plus the sum of all n_j^2. With a good first-level function the expected total is less than 2n, so the space used is **O(n)**, that is, linear.

Universal hashing

The hash functions are chosen at random from a **universal family**. A universal family is a collection of hash functions such that, for any two different keys, the number of functions in which they collide is at most (number of functions) / m. So a randomly chosen function makes two keys collide with probability at most 1/m. An example of such a family is h(k) = ((a k + b) mod p) mod m, where p is a prime larger than every key, and a, b are chosen at random with a not equal to 0.

Advantages of perfect hashing	Disadvantages of perfect hashing
Search takes O(1) time in the **worst case**	Works only for a **static** set of keys
No collisions at the time of search	Keys cannot be inserted or deleted afterwards
Space is only O(n)	Building the table takes extra time and needs a suitable hash function
Ideal for read-only data	Two-level structure is more complex than simple hashing

Applications: reserved words table of a compiler, fixed dictionaries and spelling checkers, command tables, read-only databases.

6. Summary and Formulas

Method	Idea	Search time	Limitation
Direct-address table	Key is the index	O(1) worst case	Needs one slot for every possible key
Hash table with chaining	h(k) gives the slot; a list holds the colliding keys	O(1 + alpha) average	Extra memory for pointers
Open addressing	Probe other slots of the same table	About 1/(1 - alpha) probes	alpha at most 1; deletion is difficult; clustering
Perfect hashing	Two levels, no collision	O(1) worst case	Only for static keys

Formulas to remember

Name	Formula
Load factor	alpha = n / m
Division method	h(k) = k mod m
Multiplication method	h(k) = floor(m x (kA mod 1)), 0 < A < 1
Linear probing	h(k, i) = (h'(k) + i) mod m
Quadratic probing	h(k, i) = (h'(k) + c1 i + c2 i^2) mod m
Double hashing	h(k, i) = (h1(k) + i x h2(k)) mod m
Expected probes, unsuccessful search (open addressing)	1 / (1 - alpha)
Expected probes, successful search (open addressing)	(1 / alpha) x ln(1 / (1 - alpha))
Chaining, expected search time	O(1 + alpha)
Perfect hashing, secondary table size	n_j^2

7. Important Questions for the Exam

1. What is hashing? Explain the terms hash function, hash table, collision and load factor. (5 marks)
2. What is a direct-address table? Explain its operations, advantages and disadvantages. (5 marks)
3. What is a hash table? What are the characteristics of a good hash function? (5 marks)
4. Explain the division method and the multiplication method of hashing with examples. (5 marks)
5. What is a collision? Explain collision resolution by chaining with an example. (5 marks)
6. Explain open addressing. Why is a deleted slot marked as DELETED? (5 marks)
7. Explain linear probing with an example. What is primary clustering? (5 marks)
8. Explain quadratic probing with an example. What is secondary clustering? (5 marks)
9. Explain double hashing with an example. (5 marks)
10. Compare linear probing, quadratic probing and double hashing. (5 marks)

11. Compare chaining and open addressing. (5 marks)
12. What is perfect hashing? Explain the two-level scheme with an example. (10 marks)
13. Insert the keys 89, 18, 49, 58, 69 in a table of size 10 using linear probing and quadratic probing. (5 marks)
14. What is load factor? What is rehashing? (3 marks)

Points to remember

1. Direct-address table: key = index, O(1), but too much memory.
2. Hash table: smaller table + hash function; collisions must be handled.
3. Chaining: linked list in each slot; alpha can exceed 1.
4. Open addressing: no lists, keys inside the table, alpha at most 1, deleted slots are marked.
5. Linear probing has primary clustering; quadratic has secondary clustering; double hashing has neither.
6. In double hashing, h2(k) is never 0 and m is preferably prime.
7. Perfect hashing: static keys, two levels, secondary table size n_j^2, worst case O(1), space O(n).